



Experiment No. 5

**Title: Computing probability of sentence using
Bi-grams**



Batch: NLP-1

Experiment No.: 05

Roll No.: 16010422234

Name: Chandana Galgali

Aim of the Experiment: To compute probability of sentence using Bi-grams

Program/Steps:

1. Implement a Bi-gram model to compute the probability of sentence
-

Output/Result:

```
import collections

corpus = [
    "A boy is playing in the park",
    "A girl is playing with a dog",
    "The dog is running in the park",
    "The boy is running after the dog"
]

sentence_to_calculate = "The boy is playing with a dog"

print("--- Preprocessing Data ---")
tokenized_corpus = [['<s>'] + s.lower().split() + ['</s>'] for s in corpus]
tokenized_sentence = ['<s>'] + sentence_to_calculate.lower().split() + ['</s>']
print("Tokenized Target Sentence:", tokenized_sentence)
print("-" * 25)

print("--- Counting Frequencies from Corpus ---")
all_words = [word for sent in tokenized_corpus for word in sent]
unigram_counts = collections.Counter(all_words)
bigram_counts = collections.Counter()
for sent in tokenized_corpus:
    bigram_counts.update(zip(sent, sent[1:]))

print(f"Unigram Count for 'boy': {unigram_counts['boy']}")
print(f"Bi-gram Count for ('is', 'playing'): {bigram_counts[('is', 'playing')]}")
print("-" * 25)
```

```

print(f"--- Calculating Probability for: '{sentence_to_calculate}' ---")
sentence_probability = 1.0
sentence_bigrams = list(zip(tokenized_sentence, tokenized_sentence[1:]))

for bigram in sentence_bigrams:
    prev_word, current_word = bigram
    bi_count = bigram_counts[bigram]
    uni_count = unigram_counts[prev_word]

    if uni_count == 0 or bi_count == 0:
        conditional_prob = 0
    else:
        conditional_prob = bi_count / uni_count

    print(f"P({current_word:^9} | {prev_word:^9}) = {bi_count} / {uni_count} = {conditional_prob:.4f}")
    sentence_probability *= conditional_prob

print("-" * 25)
print("--- Final Result ---")
print(f"The probability of the sentence is: {sentence_probability}")
print(f"P('{sentence_to_calculate}') = {sentence_probability:.6f}")

```

The screenshot shows a VS Code terminal window with the following output:

```

PS C:\Users\De11\Downloads\VII SEM\HO-NLP\LAB\EXP-5> & C:\Users\De11\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/De11/Downloads/VII SEM/HO-NLP/LAB/EXP-5/EXP-5.py"
--- Preprocessing Data ---
Tokenized Target Sentence: ['<s>', 'the', 'boy', 'is', 'playing', 'with', 'a', 'dog', '</s>']
--- Counting Frequencies from Corpus ---
Unigram Count for 'boy': 2
Bi-gram Count for ('is', 'playing'): 2
--- Calculating Probability for: 'The boy is playing with a dog' ---
P( the | <s> ) = 2 / 4 = 0.5000
P( boy | the ) = 1 / 5 = 0.2000
P( is | boy ) = 2 / 2 = 1.0000
P( playing | is ) = 2 / 4 = 0.5000
P( with | playing ) = 1 / 2 = 0.5000
P( a | with ) = 1 / 1 = 1.0000
P( dog | a ) = 1 / 3 = 0.3333
P( </s> | dog ) = 2 / 3 = 0.6667
--- Final Result ---
The probability of the sentence is: 0.005555555555555555
P('The boy is playing with a dog') = 0.005556
PS C:\Users\De11\Downloads\VII SEM\HO-NLP\LAB\EXP-5>

```

The status bar at the bottom indicates: Ln 49, Col 68 (1784 selected) | Spaces: 4 | UTF-8 | CRLF | Python | 3.13.0 | Go Live | AI Code Chat | Prettier.

Post Lab Question-Answers:

1. **Explain how the probability of a sentence is computed using a bi-gram model. Illustrate your answer with an example sentence**

Ans: A bi-gram model calculates the probability of a sentence by multiplying the probabilities of each word based only on its preceding word. This is known as the Markov assumption.

The probability of a word (w_i) given its previous word (w_{i-1}) is calculated from a text corpus using the formula:

$$P(w_i|w_{i-1}) = \text{Count of the pair } (w_{i-1}, w_i) / \text{Total count of the word } w_{i-1}$$

The final sentence probability is the product of these individual conditional probabilities.

Example:

Let's find the probability of the sentence "the cat sat" using this simple corpus:

- Corpus: "the cat sat on the mat", "the dog sat on the log"
- 1. Calculate individual probabilities:
- The probability of "cat" following "the" is: $P(\text{cat}|\text{the}) = \text{Count}(\text{the, cat}) / \text{Count}(\text{the}) = 1/4 = 0.25$ (The pair "the cat" appears 1 time; the word "the" appears 4 times).
- The probability of "sat" following "cat" is: $P(\text{sat}|\text{cat}) = \text{Count}(\text{cat, sat}) / \text{Count}(\text{cat}) = 1/1 = 1.0$ (The pair "cat sat" appears 1 time; the word "cat" appears 1 time).
- 2. Multiply the probabilities:
- $P(\text{"the cat sat"}) = P(\text{cat}|\text{the}) \times P(\text{sat}|\text{cat}) = 0.25 \times 1.0 = 0.25$

Outcomes: CO3 – Establish concept of Structure and Semantics**Conclusion (based on the Results and outcomes achieved):**

This experiment successfully demonstrated the computation of a sentence's probability using a bi-gram model. By tokenizing a corpus, counting the frequencies of unigrams (single words) and bi-grams (adjacent word pairs), we were able to implement the underlying probabilistic logic. The final probability of a target sentence was calculated by multiplying the conditional probabilities of each word given its preceding word. This practical exercise highlights how n-gram models, despite their simplicity, form a foundational concept in Natural Language Processing for quantifying the likelihood of text sequences.

References:**Books/ Journals/ Websites:**

1. Allen James, Natural Language Understanding, Benjamin Cumming, Second Edition, 1995
2. Jurafsky, Dan and Martin, James, Speech and Language Processing, Prentice Hall, 2008
3. Palash Goyal, Karan Jain, Sumit Pandey, Deep Learning for Natural Language Processing: Creating Neural Networks with Python, Apress, 2018